

LINGUAGEM ORIENTADA A OBJETOS

Prof. Rafael Tápia

rafael.h.tapia@cogna.com.br

Unidade 1 - Seção 1

CONCEITOS BÁSICOS DE ORIENTAÇÃO A OBJETOS EM PROGRAMAÇÃO

O que é Orientação a Objetos?

Orientação a Objetos (OO) é um jeito de programar pensando em como o mundo funciona de verdade.

Em vez de escrever um monte de código solto, a OO organiza tudo em **objetos**, que representam **coisas** do mundo real ou do sistema: uma pessoa, um carro, um produto, um botão na tela...

Esses **objetos** são criados a partir de **classes**, que funcionam como **moldes** ou **modelos** — elas dizem quais dados o objeto vai ter (**atributos**) e o que ele pode fazer (**métodos**).

Por que usar Orientação a Objetos?

Porque ela ajuda a escrever programas que são:



Mais organizados



Mais fáceis de reutilizar



Mais simples de manter e modificar



Melhores para trabalhar em equipe



ORIENTAÇÃO A OBJETOS É COMO CONSTRUIR UM SISTEMA COM PEÇAS DE LEGO: CADA OBJETO É UMA PEÇA COM FORMATO, COR E FUNÇÃO PRÓPRIA. VOCÊ JUNTA TUDO E MONTA ALGO MAIOR.

O Mundo é Orientado a Objetos!

Já pensou se você pudesse programar como se estivesse contando uma história com personagens?

Imagine um mundo com objetos:

- Um **carro** que tem **cor**, **marca**, **velocidade** e sabe **acelerar** ou **frear**.
- Um **aluno** que tem **nome**, **matrícula** e pode **estudar** ou **fazer prova**.

Tudo isso já existe no mundo real... e é justamente isso que a Orientação a Objetos tenta imitar!



Como surgiu a Orientação a Objetos?

Problema nos anos 60-70:

- Sistemas ficavam cada vez maiores.
- Código virava uma bagunça difícil de entender e manter (o chamado *spaghetti code* 🍝).

Solução:

- Em vez de pensar só em funções e dados soltos, que tal organizar tudo em objetos do mundo real?



Primeira linguagem OO: **Simula** (1967)



Popularizada com o **Smalltalk** nos anos 80.



Bombou de vez com **C++**, **Java** e depois com muitas outras.

```
Begin
  Class Pessoa;
  Begin
    Text nome;
    Procedure Apresentar;
    Begin
      OutText("Olá, meu nome é ");
      OutText(nome);
      OutImage;
    End;
  End;

End;

Pessoa p;

! Criação do objeto
p :- New Pessoa;
p.nome :- Copy("Maria");

Call p.Apresentar;
End;
```



```
Object subclass: #Pessoa
  instanceVariableNames: 'nome'.
```

```
Pessoa >> nome: umNome
  nome := umNome.
```

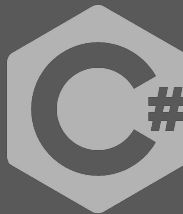
```
Pessoa >> apresentar
  Transcript show: 'Olá, meu nome é ', nome; cr.
```

```
| p |
p := Pessoa new.
p nome: 'Maria'.
p apresentar.
```





```
public class Pessoa {  
    private String nome;  
  
    public Pessoa(String nome) {  
        this.nome = nome;  
    }  
  
    public void apresentar() {  
        System.out.println("Olá, meu nome é " + nome);  
    }  
}  
  
public static void main(String[] args) {  
    Pessoa p = new Pessoa("Maria");  
    p.apresentar();  
}
```



```
public class Pessoa
{
    private string nome;

    public Pessoa(string nome)
    {
        this.nome = nome;
    }

    public void Apresentar()
    {
        Console.WriteLine("Olá, meu nome é " + nome);
    }
}

public static void Main(string[] args)
{
    Pessoa p = new Pessoa("Maria");
    p.Apresentar();
}
```

Por que usar OO?

- ✓ Ajuda a organizar o código melhor.
- ✓ Deixa mais parecido com o mundo real.
- ✓ Facilita reuso e manutenção.
- ✓ Permite trabalhar em equipe com partes independentes.

Por que às vezes NÃO usar OO?

- ✗ Pode ser exagero para programas pequenos.
- ✗ OO mal feita pode dificultar mais do que ajudar.
- ✗ Algumas tarefas funcionam melhor com abordagens funcionais ou imperativas (ex: scripts simples, processamento matemático puro, etc.)

Linguagens e OO: Quem é, quem não é, quem pode ser?

Linguagem	É OO?	Observação
Java	Sim, 100% OO	Até tudo é objeto!
C#	Sim, 100% OO	Criada pela Microsoft; parecida com Java, moderna e poderosa.
C++	Parcialmente	Suporta OO, mas permite programação procedural.
Python	Sim, mas flexível	Você pode usar OO ou não.
JavaScript	Sim (baseado em protótipos)	OO diferente (prototipal), mas dá para usar classes.
C	Não	Apenas procedural.
Go	Não é OO clássico	Usa structs e interfaces, mas sem herança.
Ruby	Sim, totalmente OO	Tudo é objeto, até números!

A Origem do Java:

A Lenda da Cafeteira Sagrada



A Origem do Java:



- Criado em **1991** por engenheiros rebeldes da **Sun Microsystems**
- Linguagem mágica para fazer aparelhos domésticos inteligentes
- Liderados por James Gosling, conhecido como o "pai do Java"
- **Primeiro nome foi Oak** (carvalho, por causa de uma árvore que tinha na frente do escritório) - mas o nome já era usado.
- **Java**, inspirado no café forte que eles tomavam litros por dia. Sim, Java é literalmente uma homenagem ao café!
- **"Write Once, Run Anywhere"** - Java Virtual Machine (JVM) - executado em qualquer sistema operacional – Windows, Mac, Linux, até em geladeiras.

Java Ganha o Mundo

- 1995 - Lançamento oficial
- 2000, Java era tipo o arroz com feijão do mundo corporativo: todo sistema grande usava.
- Java e a Briga dos Gigantes - Em 2010, a **Oracle** comprou a **Sun Microsystems**.
- Java seguiu firme, mesmo com o surgimento de linguagens rivais como C#, Python e os "queridinhos modernos" como Kotlin e Go.

E Hoje?

- Java continua vivíssimo, pode parecer meio tradicional, meio "*old school*", mas é absurdamente competente, e ainda dá aula para muita linguagem nova por aí.
- Ainda roda bilhões de dispositivos no mundo, incluindo a maior parte dos sistemas bancários, corporativos e Androids.
- Android? Vamos ver como o Java se meteu nessa história de celular:

Java + Android: um caso de casamento (quase) feliz

Quando a Google começou a desenvolver o sistema Android, lá por volta de 2005, eles queriam uma linguagem que fosse:

- Popular
- Madura
- Bem documentada
- E que **milhões de desenvolvedores já conhecessem**

A resposta? **Java**, é claro!

Então, Android é Java?

Sim, mas com um detalhe importante: ele não roda em uma **JVM padrão**, ele tem sua própria **máquina virtual do Google**. Ou seja: o código era escrito em Java, mas rodava em um **ambiente feito sob medida** para dispositivos móveis.

Java + Android:

O processo judicial (sim, teve treta)

A **Oracle**, que herdou o Java da **Sun**, processou o Google por usar partes da API do Java sem pagar licença. O processo durou de **2010 até 2021** e terminou com o **Supremo Tribunal dos EUA** decidindo que o uso da API foi "**uso justo**", com **vitória do Google**. A disputa não foi pelo uso do Java, mas pela "cópia" do código da JVM para criar a nova máquina virtual **Dalvik**.



Por fim, entra o Kotlin:

Com o tempo, a Google decidiu apoiar uma nova linguagem chamada Kotlin (em 2017), mais moderna e expressiva, também compatível com o ecossistema Java. Hoje, o Kotlin é a linguagem oficial recomendada para Android — mas Java ainda é amplamente usado.



**USAREMOS
JAVA!**

Por que vamos usar Java?



🤝 **Porque o mercado ama!**

Java é tipo aquele cantor das antigas que ainda lota estádio.

Empresas, bancos, sistemas de back-end... todo mundo ainda usa.

Por que vamos usar Java?



 Porque é um ótimo alicerce!

Java é forte em Orientação a Objetos (OO).

Aprender a programar do jeito certo, **organizando o código como se fosse montar um exército de classes e objetos super organizados.**

Aprender Java é como aprender a dirigir num carro com marcha: pode parecer mais difícil no começo, mas depois **você pilota qualquer coisa.**

Por que vamos usar Java?



🧹 Porque é limpo, organizado e cheio de ferramentas!

O código em Java é como aquela planilha bem feita do financeiro: **organizado, indentado, documentado e impossível de quebrar (ou quase).**

Além disso, o Java tem um **ecossistema gigantesco**: IDEs potentes (como IntelliJ e Eclipse), milhares de bibliotecas, frameworks, e **comunidade gigante pronta pra te ajudar** (e reclamar também, como todo bom programador).

Java

**Java
Script**

Java x JavaScript: NADA A VER!



Java	JavaScript
Criado pela Sun Microsystems (1995)	Criado pela Netscape (1995)
Compilado	Interpretado
Fortemente tipado	Fracamente tipado
Linguagem para aplicativos robustos	Usado no navegador, web, scripts
100% OO	OO baseado em protótipos (não tem classes)

Java x JavaScript

Por que os nomes são parecidos?

Foi uma jogada de marketing! A Netscape quis surfar na onda do sucesso do Java, que estava bombando na época.



TESTE DE CONHECIMENTO:

Java é o quê?

- Uma linguagem orientada a objetos 
- Uma ilha? 
- Um café? 
- Irmã do JavaScript? 

CONCEITOS FUNDAMENTAIS DA ORIENTAÇÃO A OBJETOS

CLASSE: O MOLDE

💡 **Pense numa forma de biscoito.**

Ela serve para fazer vários biscoitos com o mesmo formato.

Uma classe é como uma fôrma ou molde. Ela define como os objetos daquele tipo serão: quais informações vão ter (atributos) e o que vão poder fazer (métodos).

Exemplo:

Classe **Pessoa**:

Atributos: nome, idade, endereço

Métodos: falar(), andar()



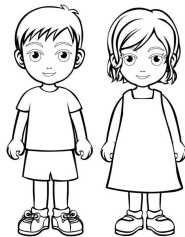
OBJETO: O BISCOITO PRONTO

🍪 Cada biscoito feito com a mesma forma pode ter coberturas diferentes, tamanhos diferentes, mas todos seguem o mesmo formato básico.

Um **objeto** é uma **coisa real** feita com base na classe. Ele tem seus próprios dados e pode agir.

Exemplo:

```
joao = nova Pessoa()  
maria = nova Pessoa()
```



Joao e Maria são objetos (ou instâncias) da classe Pessoa. Cada um tem seu nome, sua idade, seu endereço.

ATRIBUTO: AS CARACTERÍSTICAS

 Se um objeto fosse um personagem de jogo, os atributos seriam sua força, agilidade, vida...

Atributos são as **informações que um objeto carrega**. Eles descrevem as suas características.

Exemplo:

Na classe Pessoa, os atributos podem ser:

- nome = "João"
- idade = 25
- endereço = "Rua das Flores"

MÉTODO: O QUE O OBJETO FAZ

 Um método é uma **ação** que o objeto sabe fazer. É como apertar um botão no controle do personagem.

Métodos são **funções** dentro da classe. Eles definem o comportamento do objeto — ou seja, o que ele pode fazer.

Exemplo:

Na classe Pessoa, os métodos podem ser:

- **falar()**: mostra uma mensagem no console.
- **andar()**: muda a posição da pessoa no mapa.

INSTÂNCIA: O NASCIMENTO DO OBJETO

 Quando usamos o molde (classe) para “fabricar” um objeto, estamos criando uma instância.

Criar uma instância é o mesmo que criar um objeto a partir da classe.

Exemplo:

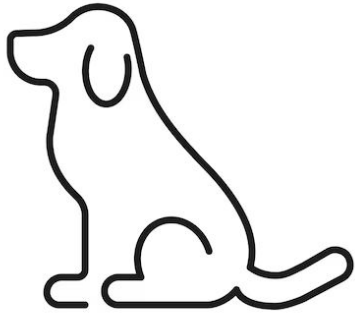
```
Pessoa maria = new Pessoa();
```

RESUMO:

Conceito	Explicação Simples	Analogia
Classe	Molde que define o que o objeto será	Forma de biscoito 🍪
Objeto	Instância real baseada na classe	O biscoito pronto
Atributo	Informações sobre o objeto	Cobertura, sabor, tamanho
Método	Ações que o objeto pode fazer	“Comer”, “Empacotar”, “Quebrar”
Instância	Processo de criar um objeto a partir da classe	Biscoito saindo da forma 🍴🍴

EXEMPLOS

CLASSE CACHORRO



Características / Atributos:

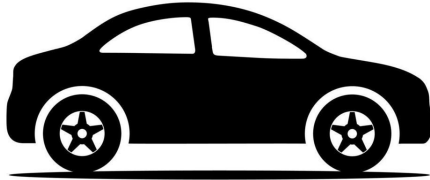
nome, qtde_comida, sono, fala

Funções / Métodos:

comer(), dormir(), falar()



CLASSE CARRO



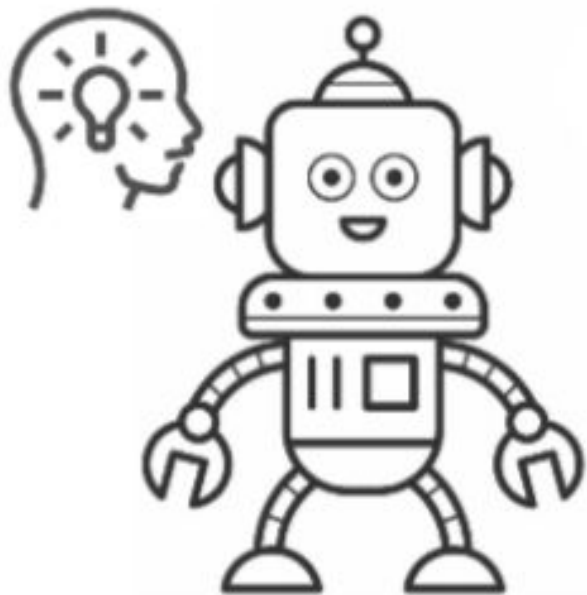
Características / Atributos:

placa, montadora, modelo, marca,
ano, cor, quilometragem

Funções / Métodos:

ligar(), desligar()

CLASSE ROBOT



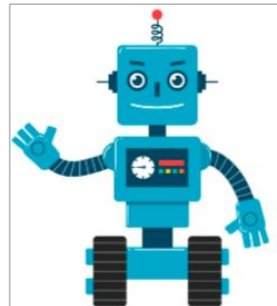
Classe

atributos:

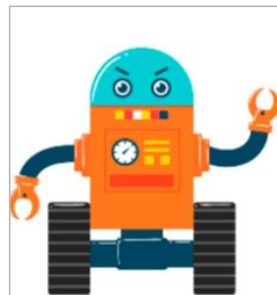
nome
cor
velocidade_max
nivel_bateria_atual
peso_carga_max
tipo_tracao
tem_antena

métodos:

mover(direcao)
pegar_caixa(localizacao)
entregar_caixa(localizacao)
falar(mensagem)



Objeto



Objeto

OS ATRIBUTOS PODEM MUDAR



No jogo **Super Mario Bros**, temos a classe **Personagem**, que serve como **molde** para criar personagens do jogo. Ela define os **atributos** e **métodos** que qualquer personagem pode ter.

Durante o jogo, só temos um personagem jogável, o Mario — ou seja, **apenas um objeto** da classe Personagem. Esse objeto muda de estado ao longo do tempo: seus atributos (como forma) são alterados com base em eventos do jogo (pegar cogumelo, flor de fogo, levar dano...).

MÉTODOS

MÉTODOS: AÇÕES QUE OS OBJETOS SABEM FAZER

Em programação orientada a objetos, métodos são as ações que um objeto pode realizar.

Exemplo: se temos um objeto carro, ele pode:

- ligar o motor
- andar
- abastecer
- verificar o combustível

Essas ações são representadas por métodos dentro da classe.

MÉTODOS SEM PARÂMETRO E SEM RETORNO

O que é?

Um método **sem parâmetro** e **sem retorno** não precisa de informação externa e não devolve nenhum resultado.

Exemplo:

```
void ligarMotor() {  
    System.out.println("Motor ligado!");  
}
```

Uso:

```
carro.ligarMotor();
```

Na linguagem Java, “void” é uma palavra usada para dizer que um método não retorna nenhum valor.

MÉTODOS COM PARÂMETRO E SEM RETORNO

O que é?

Esse tipo de método precisa de **dados de entrada (parâmetro)**, mas não retorna resultado.

Exemplo:

```
void abastecer(double litros) {  
    quantidade_combustivel += litros;  
}
```

Uso:

```
carro.abastecer(20.0);
```

MÉTODOS SEM PARÂMETRO E COM RETORNO

O que é?

Esse método não recebe dados, mas informa algo de dentro do objeto.

Exemplo:

```
double verificarCombustivel() {  
    return quantidade_combustivel;  
}
```

Uso:

```
double nivel = carro.verificarCombustivel();  
System.out.println("Temos " + nivel + " litros.");
```

MÉTODOS COM PARÂMETRO E COM RETORNO

O que é?

O método usa **dados de entrada (parâmetros)**, faz um cálculo ou ação, e **retorna um resultado**.

Exemplo:

```
double calcularAutonomia(double kmPorLitro) {  
    return quantidade_combustivel * kmPorLitro;  
}
```

Uso:

```
double autonomia = carro.calcularAutonomia(12.5);  
System.out.println("Autonomia estimada: " + autonomia + " km");
```

EXEMPLOS DE CÓDIGOS JAVA

```
public static void main(String[] args) {  
    String nome1 = "João da Silva";  
    String cpf1 = "123.456.789-10";  
    String dataNascimento1 = "01/01/2000";  
  
    String nome2 = "Maria Souza";  
    String cpf2 = "987.654.321-00";  
    String dataNascimento2 = "15/03/1995";  
  
    System.out.println("Nome: " + nome1);  
    System.out.println("CPF: " + cpf1);  
    System.out.println("Data de Nascimento: " + dataNascimento1);  
  
    System.out.println("Nome: " + nome2);  
    System.out.out.println("CPF: " + cpf2);  
    System.out.println("Data de Nascimento: " + dataNascimento2);  
}
```

```
class Pessoa {
    String nome;
    String cpf;
    String dataNascimento;

    // Construtor
    public Pessoa(String nome, String cpf, String dataNascimento) {
        this.nome = nome;
        this.cpf = cpf;
        this.dataNascimento = dataNascimento;
    }

    // Método para exibir as informações da pessoa
    public void exibirInformacoes() {
        System.out.println("Nome: " + nome);
        System.out.println("CPF: " + cpf);
        System.out.println("Data de Nascimento: " + dataNascimento);
    }
}
```

```
public static void main(String[] args) {  
    // Criando objetos da classe Pessoa  
    Pessoa pessoa1 = new Pessoa("João da Silva", "123.456.789-10", "01/01/2000");  
    Pessoa pessoa2 = new Pessoa("Maria Souza", "987.654.321-00", "15/03/1995");  
  
    // Chamando o método para exibir as informações  
    pessoa1.exibirInformacoes();  
    pessoa2.exibirInformacoes();  
}
```

SOBRE CÓDIGO

Programar não é só fazer o código funcionar...

- É fazer o código ser compreensível, reutilizável e mantido por outras pessoas (***ou até por você mesmo no futuro, quando nem lembrar mais o que escreveu***).
- É fazer o código fazer sentido para quem vai mexer nele depois.





O MELHOR CÓDIGO NÃO É O MAIS INTELIGENTE.
É O MAIS FÁCIL DE ENTENDER!



**MONTANDO
NOSSO
AMBIENTE**

JDK (Java Development Kit)

É o kit de desenvolvimento Java. Contém tudo o que você precisa para criar, compilar e executar programas em Java:

- **Compilador** (javac) → transforma código .java em bytecode .class.
- **JRE** (Java Runtime Environment) → para executar programas Java.
- **Ferramentas extras** (depuradores, geradores de documentação, etc.).

Site:

<https://www.oracle.com/java/technologies/javase/jdk21-archive-downloads.html>

Download:

https://download.oracle.com/java/21/archive/jdk-21.0.7_windows-x64_bin.msi

ECLIPSE

É um IDE (**Integrated Development Environment**) — ou seja, um programa para escrever código com mais produtividade.

No Eclipse você pode:

- Criar projetos Java de forma organizada.
- Ter autocompletar, depuração, atalhos e plugins.
- Compilar e rodar seu programa com um clique, sem precisar digitar comandos no terminal.

Site:

<https://www.eclipse.org/downloads/>

Download:

https://www.eclipse.org/downloads/download.php?file=/oomph/epp/2025-06/R/eclipse-inst-jre-win64.exe&mirror_id=576

LOMBOK

É uma biblioteca Java que reduz a quantidade de código repetitivo (boilerplate). Com anotações como @Getter, @Setter e @Builder, o Lombok gera automaticamente métodos que você teria que escrever manualmente (getters, setters, construtores, etc.), deixando o código mais limpo.

Site:

<https://projectlombok.org/>

Download:

<https://projectlombok.org/downloads/lombok.jar>

Fim da transmissão!

- Lembrem-se de fazer as outras atividades do AVA
- Todo o material da aula está/será disponibilizado no AVA
- Não falem às aulas
- Algumas atividades valendo pontos serão feitas em sala
- Dúvidas: rafael.h.tapia@cogna.com.br